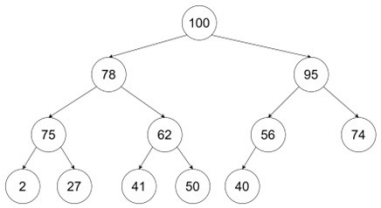


Heaps

1

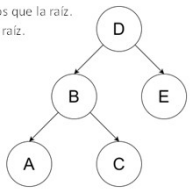
Repaso

2

El BinarySearchTree ADT

¿Qué es un BinarySearchTree (BST)?

- Es un Binary Tree desequilibrado con estas propiedades:
  - Todos los nodos del subárbol de la izquierda son más pequeños que la raíz.
  - Todos los nodos del subárbol derecho son más grandes que la raíz.
- En otras palabras, para cualquier nodo N:
  - Todos los descendientes de la izquierda son menores que N.
  - Todos los descendientes de la derecha son mayores que N.



3

Heaps

4

Heaps

Los heaps no son árboles de uso general—no se usan para buscar o desplazarse como los BST.

Están especializados para una cosa: Obtener (y actualizar) eficientemente el elemento de mayor o menor prioridad.

Caso de uso principal: filas de prioridad (razón principal por la que existen).

Casos de uso:

- Planificación de tareas (ejecuta primero el trabajo más urgente)
- filas de impresión (primero documentos de alta prioridad)
- Programación de CPU/procesos (qué se va a hacer primero)

5

5

Heaps

- Un árbol binario completo que sigue a la propiedad del heap (propiedad heap-order).
  - Un árbol binario completo es un tipo de árbol binario en el que todos los niveles, excepto posiblemente el último, están completamente llenos y todos los nodos están lo más a la izquierda posible.
- Al igual que un BST, este es un árbol ordenado.

6

6

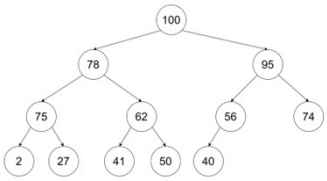
Heaps

- Dos posibilidades:
  - el key de un nodo debe ser mayor o igual que sus hijos.
    - Esto se conoce como un **Max Heap**.
    - Tenga en cuenta que con esta regla la raíz será el valor más grande.
  - el key de un nodo debe ser menor o igual que sus hijos.
    - Esto se conoce como un **Min Heap**.
    - Tenga en cuenta que con esta regla la raíz será el valor más pequeño.

7

7

Ejemplo de un Heap



Esto es un Max Heap.

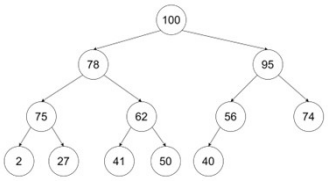
Notar:

- Cada hijo es más pequeño que su padre.
- La raíz es el key más grande.
- Cada subárbol es también un heap.
- La organización es diferente a la BST.
- Este es un árbol completo.

8

8

Ejemplo de un Heap



9

Min Heaps

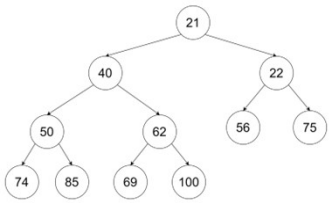
¿Qué sería un Min Heap?

10

Min Heap

Es un árbol donde el key de cada nodo es más pequeño que sus hijos.

- La raíz es el key más pequeño.
- Cada elemento secundario es más grande que su elemento primario.



11

Max Heaps

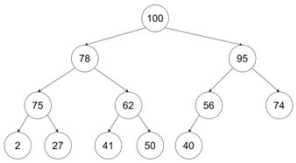
¿Qué sería un Max Heap?

12

Max Heap

Es un árbol donde el key de cada nodo es más grande que sus hijos.

- La raíz es el key más grande.
- Cada elemento secundario es más pequeño que su elemento primario.



13

Heap ADT

14

Heap Operaciones

METHOD	DESCRIPTION
void add(Entry<K, V> e)	Agrega elemento al heap mientras se mantiene la propiedad del heap.
Entry<K, V> removeMax()	Quita y devuelve el elemento más grande del heap.
Entry<K, V> getMax( )	Devuelve el elemento más grande del heap.

El ADT también tiene los métodos típicos, tales como: `isEmpty()`, `size()`, y `clear()`.

Tenga en cuenta que estos métodos son para implementar un Max Heap.

¿En qué se diferenciarían los métodos remove() y get() para un Min Heap?

15

Inserción en el Heap

¿Cómo añadimos a un heap?

Al fin y al cabo, tenemos que mantener la coherencia del heap. Esto significa que después de cada add()el heap:

- Debe seguir siendo un árbol completo.
- Cada nodo debe seguir la propiedad del heap.

16

Insertar en elHeap

¿Cómo lo hacemos?

17

17

Insertar en el Heap (Max)

¿Cómo lo hacemos?

- Podríamos simplemente agregar el nuevo elemento en el nivel final para mantener el Árbol completo.
- Pero, ¿qué sucede si el nuevo elemento es más grande que su nodo principal? En ese caso, la propiedad del heap no se mantendría.
- Para solucionar esto, tendríamos que arreglar el árbol después de agregar el nodo. Este proceso se denomina "sifting up" o "up-head bubbling".

18

18

Insertar en el Heap

¿Cómo lo hacemos?

- El proceso de sifting up consiste en intercambiar de forma recursiva la nueva entrada con su elemento primario hasta que se restaure la propiedad del heap.
- En resumen, primero agregamos el nodo y luego arreglamos el árbol.

19

19

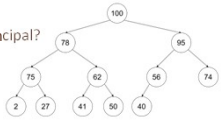
Insertar en el Heap

¿Cómo lo hacemos??

- Más detalladamente, estos son los pasos para un Max Heap:
  - Inserte el nodo en su posición correspondiente (para mantener el árbol completo))
  - Si bien el nodo no es la raíz y es más grande que su padre, intercambiamos el nodo con su padre.

¿Cuál sería la posición correspondiente?

¿Por qué funciona el intercambio de la nueva entrada y la principal?

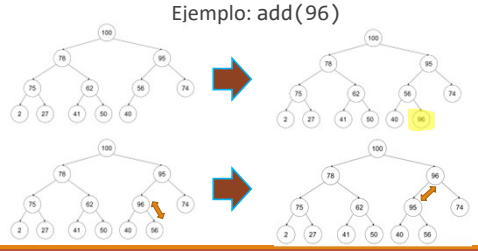


20

20

Insertando en el Heap

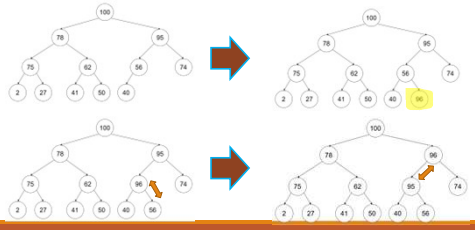
Ejemplo: add(96)



21

Insertando en elHeap

Ejemplo 1: add(96)



Notar:

- Al agregar en el último nivel, garantizamos que el árbol estará completo.
- Dejamos de intercambiar una vez que se restaura la propiedad del heap.
- En el peor de los casos, la nueva entrada llega a la parte superior y se convierte en la nueva entrada máxima.

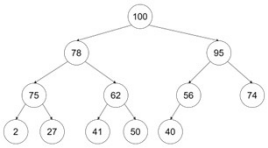
22

Quitar de un Heap

- Cuando removemos de un heap, quitamos la raíz.
- Como en un Stack o Queue, Solo tenemos acceso a un valor específico.
- Pero hacerlo nos dejaría con dos subárboles, ¿cómo mantenemos la coherencia del heap?

¿Cómo manejamos la eliminación en un BST?

¿Funcionaría aquí?



23

Quitar de un Heap

- Al igual que en los BST, la idea aquí sería reemplazar el nodo eliminado con uno de sus descendientes.
- Para mantener el árbol completo, elegimos el último nodo del árbol.
- Pero esto potencialmente violará la propiedad del heap, entonces, ¿qué hacemos?
- Igual que antes, arreglamos el heap.
- En este caso aplicamos sifting down (o down-heap bubbling).
- Como habrás imaginado, ahora cambiamos hacia abajo hasta que se restaura la propiedad del heap.

24

Quitar de un Heap

En sifting up, intercambiamos nodos si la nueva entrada es más grande que su padre. Entonces,

¿Qué hijo debe reemplazar el nodo cuando aplicamos sifting down?

a) A la izquierda, siempre la izquierda.
b) A la derecha, merece más reconocimiento.
c) Cualquiera de los dos hijos servirá.
d) El hijo más grande.
e) El hijo más pequeño.

25

Quitar de un Heap

Deberíamos elegir al hijo más grande. Si lo haces, te asegurarás de que mantenemos la propiedad del heap.

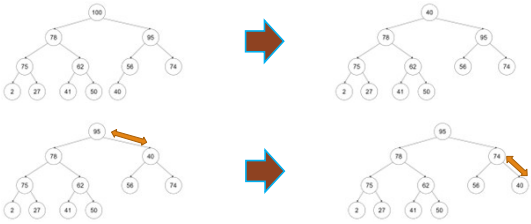
En resumen, los pasos para removeMax() son los siguientes:

- Retire la raíz y reemplácela con el último nodo del árbol.
- Si bien el nodo tiene hijos y es más pequeño que cualquiera de ellos, cámbielo con su hijo más grande.

26

Quitar de un Heap

Ejemplo: removeMax()



27

Run Time Analysis

- El peor de los casos, tanto para add() como para removeMax() sería O(h) = O(log n).
- Donde h = altura del árbol y n = nodos totales en el heap.
- Para el getMax() sería O(1), ya que solo necesitamos devolver el Entry en la raíz.

28

Análisis en tiempo de ejecución

- Tenga en cuenta que los heaps son estructuras eficientes en el caso de que queramos acceder al valor máximo (o mínimo) de una colección.
- Esta es una de las estructuras ideales para implementar un Priority Queue. la siguiente estructura que cubriremos.

29

Heap Implementación

30

Implementación Heaps

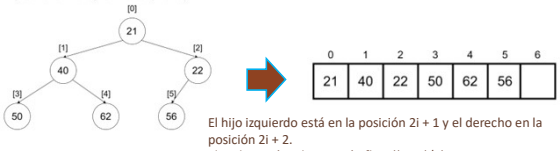
- Cuando discutimos anteriormente las implementaciones de árboles, mencionamos que podemos usar matrices.
- El truco consistía en que con una fórmula sabíamos dónde se almacenaban los hijos de cada nodo dentro del array.
- Sí, podemos implementar heaps usando matrices (o ArrayLists).

¿Por qué haríamos eso?

31

Implementación Heaps

- Debido a que el heap es un árbol completo, eso significa que llenaremos el árbol por nivel.
- En términos de la matriz, esto significa que la matriz se llenará en la última posición disponible (nula).

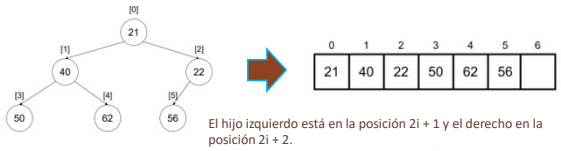


El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2)

32

Implementación Heaps

- Esto facilita la implementación de operaciones como add() ya que cuando agregamos un nuevo elemento, iría en posición **`size`**.
- El intercambio es una cuestión de comparación con el padre en la posición floor((i – 1)/2).



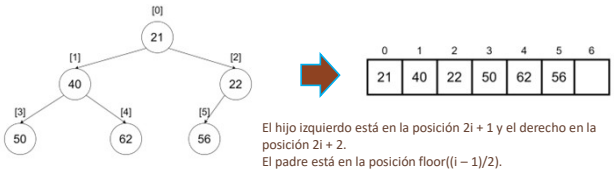
El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

33

33

Implementación Heaps

- Del mismo modo, para removeMin(), encontramos el reemplazo en la posición **`size – 1`**.
- El intercambio lo comparamos con los hijos en las posiciones 2i + 1 (izquierda) y 2i + 2 (derecha).



El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

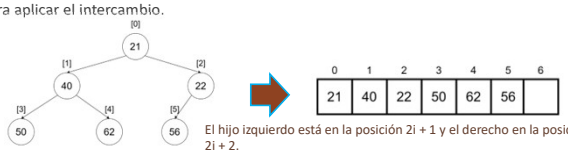
34

34

Implementación Heaps

Si implementamos el heap usando ArrayLists:

- ¿Qué métodos usaríamos para el add()?
  - Para agregar el nodo
  - Para aplicar el intercambio.



El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

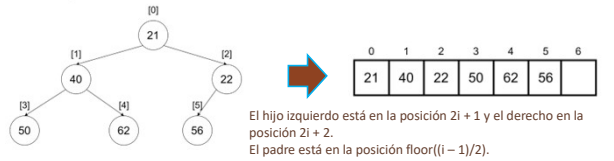
35

35

Implementación Heaps

Si implementamos el heap usando ArrayLists:

- ¿Qué métodos usaríamos para el add()?
  - Para agregar el nodo. Llamamos a add().
  - Para aplicar el intercambio. Llamamos a set() para el padre y el nodo.



El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

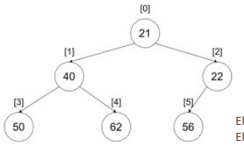
36

36

Implementación Heaps

Si implementamos el heap usando ArrayLists:

- ¿Qué métodos usaríamos para el removeMin()?
  - Para reemplazar la raíz. Usamos set() para hacer el reemplazo y remove(size-1)
  - Para aplicar el intercambio. Usamos el método set()



0	1	2	3	4	5	6
21	40	22	50	62	56	

El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

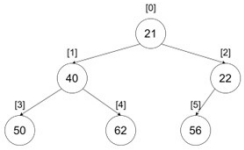
37

37

Implementación Heaps

Si implementamos el heap usando ArrayLists:

- ¿Qué métodos usaríamos para el removeMax()?
  - Para reemplazar la raíz. Usamos set() para hacer el reemplazo y remove(size-1).
  - Para aplicar el intercambio. Usamos el método set().



0	1	2	3	4	5	6
21	40	22	50	62	56	

El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

38

38

Heaps

En Java:

- java.util.PriorityQueue
- ArrayLists

39

39

Implementación usando ArrayLists

Método	Descripción	Ejemplo
add(E e)	Añade elemento al final	list.add("A");
add(int index, E e)	Inserta elemento en posición	list.add(1, "B");
get(int index)	Devuelve el elemento en el índice	list.get(0);
set(int index, E e)	Reemplaza el elemento en el índice	list.set(0, "X");
remove(int index)	Elimina elemento en el índice	list.remove(0);
remove(Object o)	Elimina la primera aparición	list.remove("A");
size()	Devuelve el número de elementos	list.size();
isEmpty()	Comprueba si la lista está vacía	list.isEmpty();
contains(Object o)	Comprueba si existe un elemento	list.contains("A");
clear()	Elimina todos los elementos	list.clear();
indexOf(Object o)	Devuelve el primer índice del elemento	list.indexOf("A");
lastIndexOf(Object o)	Retorna el índice de última ocurrencia	list.lastIndexOf("A");

40

Implementación Heaps

- ¿Por qué no usar nodos en su lugar?
- Tenga en cuenta que podemos usar nodos para implementar un heap, pero el uso de matrices hace que sea más sencillo de implementar.
- Con la matriz:
  - Aproveche mejor su propiedad completa. Hace que la búsqueda de nodos sea O(1).
  - Ahorra espacio en la memoria. No se necesitan punteros.

41

41

Implementación Heaps

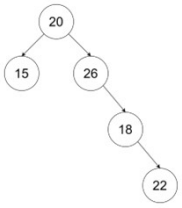
- ¿Por qué no usar matrices para todos los árboles??
- Un árbol binario normal podría desperdiciar más espacio usando una matriz al tener demasiados espacios vacíos.
- Dado que los árboles regulares no siempre están completos, eso significa que podría estar muy desequilibrado, lo que resultaría en que la matriz tuviera datos muy escasamente colocados.

42

42

implementing Heaps

- Por ejemplo, ¿cuál sería la matriz para este árbol**?**



El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

43

43

Implementación Heaps

- Por ejemplo, ¿cuál sería la matriz para este árbol?

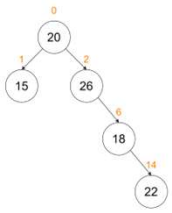
El hijo izquierdo está en la posición 2i + 1 y el derecho en la posición 2i + 2.
El padre está en la posición floor((i – 1)/2).

44

44

Implementación Heaps

- Por ejemplo, ¿cuál sería la matriz para este árbol?



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
20	15	26				18								22	

45

Conclusión

46

Conclusión

- Vimos un tipo diferente de árbol.
- Al igual que el BST, este tiene una forma específica de ser ordenado.
- Dado que es un árbol completo, podemos beneficiarnos de implementarlo usando matrices.

47